A Little C Primer/C File-IO Through Library Functions

The file-I/O library functions are much like the console-I/O functions. In fact, most of the console-I/O functions can be thought of as special cases of the file-I/O functions. The library functions include:

```
fopen()      Create or open a file for reading or writing.
fclose()     Close a file after reading or writing it.

fseek()      Seek to a certain location in a file.
rewind()     Rewind a file back to its beginning and leave it open.
rename()     Rename a file.
remove()     Delete a file.

fprintf()    Formatted write.
fscanf()     Formatted read.
fwrite()     Unformatted write.
fread()      Unformatted read.
putc()       Write a single byte to a file.
getc()       Read a single byte from a file.
fputs()      Write a string to a file.
fgets()      Read a string from a file.
```

All these library functions depend on definitions made in the "stdio.h" header file, and so require the declaration:

```
#include <stdio.h>
```

C documentation normally refers to these functions as performing "stream I/O", not "file I/O". The distinction is that they could just as well handle data being transferred through a modem as a file, and so the more general term "data stream" is used rather than "file". However, we'll stay with the "file" terminology in this document for the sake of simplicity.

The "fopen()" function opens and, if need be, creates a file. Its syntax is:

```
<file pointer> = fopen( <filename>, <access mode> );
```

The "fopen()" function returns a "file pointer", declared as follows:

```
FILE *<file pointer>;
```

The file pointer will be returned with the value NULL, defined in "stdio.h", if there is an error. The "access modes" are defined as follows:

```
r     Open for reading.
w     Open and wipe (or create) for writing.
```

```
    a     Append -- open (or create) to write to end of file.
    r+    Open a file for reading and writing.
    w+    Open and wipe (or create) for reading and writing.
    a+    Open a file for reading and appending.
```

The "filename" is simply a string of characters.

It is often useful to use the same statements to communicate either with files or with standard I/O. For this reason, the "stdio.h" header file includes predefined file pointers with the names "stdin" and "stdout". There's no need to do an "fopen()" on them—they can just be assigned to a file pointer:

```
    fpin = stdin;
    fpout = stdout;
```

—and any following file-I/O functions won't know the difference.

The "fclose()" function simply closes the file given by its file pointer parameter. It has the syntax:

```
    fclose( fp );
```

The "fseek()" function call allows the byte location in a file to be selected for reading or writing. It has the syntax:

```
    fseek( <file_pointer>, <offset>, <origin> );
```

The offset is a "long" and specifies the offset into the file, in bytes. The "origin" is an "int" and is one of three standard values, defined in "stdio.h":

```
    SEEK_SET    Start of file.
    SEEK_CUR    Current location.
    SEEK_END    End of file.
```

The "fseek()" function returns 0 on success and non-zero on failure.

The "rewind()", "rename()", and "remove()" functions are straightforward. The "rewind()" function resets an open file back to its beginning. It has the syntax:

```
    rewind( <file_pointer> );
```

The "rename()" function changes the name of a file:

```
    rename( <old_file_name_string>, <new_file_name_string> );
```

The "remove()" function deletes a file:

```
    remove( <file_name_string> )
```

The "fprintf()" function allows formatted ASCII data output to a file, and has the syntax:

```
fprintf( <file pointer>, <string>, <variable list> );
```

The "fprintf()" function is identical in syntax to "printf()", except for the addition of a file pointer parameter. For example, the "fprintf()" call in this little program:

```
/* fprpi.c */

#include <stdio.h>

void main()
{
  int n1 = 16;
  float n2 = 3.141592654f;
  FILE *fp;

  fp = fopen( "data", "w" );
  fprintf( fp, "  %d   %f", n1, n2 );
  fclose( fp );
}
```

—stores the following ASCII data:

```
 16   3.14159
```

The formatting codes are exactly the same as for "printf()":

```
%d    decimal integer
%ld   long decimal integer
%c    character
%s    string
%e    floating-point number in exponential notation
%f    floating-point number in decimal notation
%g    use %e and %f, whichever is shorter
%u    unsigned decimal integer
%o    unsigned octal integer
%x    unsigned hex integer
```

Field-width specifiers can be used as well. The "fprintf()" function returns the number of characters it dumps to the file, or a negative number if it terminates with an error.

The "fscanf()" function is to "fprintf()" what "scanf()" is to "printf()": it reads ASCII-formatted data into a list of variables. It has the syntax:

```
fscanf( <file pointer>, <string>, <variable list> );
```

However, the "string" contains only format codes, no text, and the "variable list" contains the addresses of the variables, not the variables themselves. For example, the program below reads back the two numbers that were stored with "fprintf()" in the last example:

```
/* frdata.c */

#include <stdio.h>
```

```
void main()
{
  int n1;
  float n2;
  FILE *fp;

  fp = fopen( "data", "r" );
  fscanf( fp, "%d %f", &n1, &n2 );
  printf( "%d %f", n1, n2 );
  fclose( fp );
}
```

The "fscanf()" function uses the same format codes as "fprintf()", with the familiar exceptions:

- There is no "%g" format code.
- The "%f" and "%e" format codes work the same.
- There is a "%h" format code for reading short integers.

Numeric modifiers can be used, of course. The "fscanf()" function returns the number of items that it successfully read, or the EOF code, an "int", if it encounters the end of the file or an error.

The following program demonstrates the use of "fprintf()" and "fscanf()":

```
/* fprsc.c */

#include <stdio.h>

void main()
{
  int ctr, i[3], n1 = 16, n2 = 256;
  float f[4], n3 = 3.141592654f;
  FILE *fp;

  fp = fopen( "data", "w+" );

  /* Write data in:   decimal integer formats
                      decimal, octal, hex integer formats
                      floating-point formats  */

  fprintf( fp, "%d %10d %-10d \n", n1, n1, n1 );
  fprintf( fp, "%d %o %x \n", n2, n2, n2 );
  fprintf( fp, "%f %10.10f %e %5.4e \n", n3, n3, n3, n3 );

  /* Rewind file. */

  rewind( fp );

  /* Read back data. */

  puts( "" );
  fscanf( fp, "%d %d %d", &i[0], &i[1], &i[2] );
  printf( "   %d\t%d\t%d\n", i[0], i[1], i[2] );
  fscanf( fp, "%d %o %x", &i[0], &i[1], &i[2] );
  printf( "   %d\t%d\t%d\n", i[0], i[1], i[2] );
  fscanf( fp, "%f %f %f %f", &f[0], &f[1], &f[2], &f[3] );
  printf( "   %f\t%f\t%f\t%f\n", f[0], f[1], f[2], f[3] );

  fclose( fp );
}
```

The program generates the output:

```
16         16         16
256        256        256
```

```
3.141593   3.141593   3.141593   3.141600
```

The "fwrite()" and "fread()" functions are used for binary file I/O. The syntax of "fwrite()" is as follows:

```
fwrite( <array_pointer>, <element_size>, <count>, <file_pointer> );
```

The array pointer is of type "void", and so the array can be of any type. The element size and count, which give the number of bytes in each array element and the number of elements in the array, are of type "size_t", which are equivalent to "unsigned int".

The "fread()" function similarly has the syntax:

```
fread( <array_pointer>, <element_size>, <count>, <file_pointer> );
```

The "fread()" function returns the number of items it actually read.

The following program stores an array of data to a file and then reads it back using "fwrite()" and "fread()":

```
/* fwrrd.c */

#include <stdio.h>

#include <math.h>

#define SIZE 20

void main()
{
  int n;
  float d[SIZE];
  FILE *fp;

  for( n = 0; n < SIZE; ++n )                /* Fill array with roots. */
  {
    d[n] = (float)sqrt( (double)n );
  }
  fp = fopen( "data", "w+" );                /* Open file. */
  fwrite( d, sizeof( float ), SIZE, fp );    /* Write it to file. */
  rewind( fp );                              /* Rewind file. */
  fread( d, sizeof( float ), SIZE, fp );     /* Read back data. */
  for( n = 0; n < SIZE; ++n )                /* Print array. */
  {
    printf( "%d: %7.3f\n", n, d[n] );
  }
  fclose( fp );                              /* Close file. */
}
```

The "putc()" function is used to write a single character to an open file. It has the syntax:

```
putc( <character>, <file pointer> );
```

The "getc()" function similarly gets a single character from an open file. It has the syntax:

```
<character variable> = getc( <file pointer> );
```

The "getc()" function returns "EOF" on error. The console I/O functions "putchar()" and "getchar()" are really only special cases of "putc()" and "getc()" that use standard output and input.

The "fputs()" function writes a string to a file. It has the syntax:

```
fputs( <string / character array>, <file pointer> );
```

The "fputs()" function will return an EOF value on error. For example:

```
fputs( "This is a test", fptr );
```

The "fgets()" function reads a string of characters from a file. It has the syntax:

```
fgets( <string>, <max_string_length>, <file_pointer> );
```

The "fgets" function reads a string from a file until if finds a newline or grabs <string_length-1> characters. It will return the value NULL on an error.

The following example program simply opens a file and copies it to another file, using "fgets()" and "fputs()":

```
/* fcopy.c */

#include <stdio.h>


#define MAX 256

void main()
{
  FILE *src, *dst;
  char b[MAX];

  /* Try to open source and destination files. */

  if ( ( src = fopen( "infile.txt", "r" )) == NULL )
  {
     puts( "Can't open input file." );
     exit();
  }
  if ( (dst = fopen( "outfile.txt", "w" )) == NULL )
  {
     puts( "Can't open output file." );
     fclose( src );
     exit();
  }

  /* Copy one file to the next. */

  while( ( fgets( b, MAX, src ) ) != NULL )
  {
     fputs( b, dst );
  }

  /* All done, close up shop. */

  fclose( src );
```

```
    fclose( dst );
  }
```